DAY 58 DAY 59 APACHE KAFKA RESILIENCE & RELIABILITY

Kafka Error Handling, DLQ & Idempotency

Complete Implementation Reference Notes: Dead-Letter Queue (DLQ / DLT), Exponential Backoff, Idempotent Producers, Consumer Groups & Message Deduplication.

PROJECT	PRODUCER MICROSERVICE	CONSUMER MICROSERVICE	GENERATED DATE
Digital Banking System	**Enterprise-Digital-Banking-System**	**notification-service**	**2026-10-26**

1. Architectural Big Picture

In our event-driven banking architecture, the primary banking app publishes domain events when money moves or beneficiaries activate. The `notification-service` consumes them to send SMS, Emails, or In-App alerts.

PROBLEM 1 **Message Failures (Poison Pills)**

If the notification consumer fails (e.g. SMTP down, bad JSON), standard Kafka retries indefinitely. The partition blocks, and all subsequent customers stop receiving notifications.

Solution (Day 58): Retry with **Exponential Backoff** (3-4 attempts), then route permanently failed messages to a **Dead-Letter Topic (.DLT)** without stalling the partition.

PROBLEM 2 **Duplicate Processing**

Network hiccups after publish, consumer rebalances, or server restarts cause Kafka to deliver the exact same event twice. A customer could get 2-3 emails or debited twice if actions are not idempotent.

Solution (Day 59): Enable **Producer Idempotence** (prevents duplicate writes) and implement **Consumer Deduplication** using a database/Redis unique key guard.

2. Day 58: Kafka Error Handling & Dead-Letter Queue (DLQ)

2.1 Transient Failures vs. Poison Pills

Failure Type	Typical Examples	Recommended Handling Strategy
Transient Errors RETRYABLE	• Email / SMS Gateway timeout (503) • Database connection blip • Network latency	Retry with **Exponential Backoff** (e.g. initial delay 1s, multiplier 2.0, max 3 attempts). Allow the downstream dependency time to recover.
Poison Pills NON-RETRYABLE	• Malformed JSON string • Deserialization failure • Missing required fields / NullPointerException	**Do NOT retry.** Retrying a deserialization bug 1,000 times will never fix it. Send straight to **.DLT** (Dead Letter Topic) immediately.

2.2 Approach 1: Spring Kafka `@RetryableTopic` (Modern & Declarative)

Spring Kafka provides `@RetryableTopic` which automatically creates retry topics (e.g. `transaction.completed-retry-1000`, `transaction.completed-retry-2000`) and a final dead-letter topic (`transaction.completed-dlt`).

```
Enterprise-Digital-Banking-System / kafka / consumer / TransactionEventConsumer.java                    Java
```

```java
package org.example.enterprisedigitalbankingsystem.kafka.consumer;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.example.enterprisedigitalbankingsystem.kafka.consumer.repository.ProcessedEventRepository;
import org.example.enterprisedigitalbankingsystem.kafka.event.TransactionCompletedEvent;
import org.springframework.kafka.annotation.DltHandler;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.annotation.RetryableTopic;
import org.springframework.kafka.support.KafkaHeaders;
import org.springframework.messaging.handler.annotation.Header;
import org.springframework.retry.annotation.Backoff;
import org.springframework.stereotype.Component;
import tools.jackson.databind.ObjectMapper;

@Component
@RequiredArgsConstructor
@Slf4j
public class TransactionEventConsumer {

    private static final String TOPIC = "transaction.completed";
    private final ObjectMapper objectMapper;
    private final ProcessedEventRepository processedEventRepository;

    // Day 58: Retry 3 times with exponential backoff (1s → 2s → 4s), then route to DLT
    @RetryableTopic(
        attempts = "4",
        backoff = @Backoff(delay = 1000, multiplier = 2.0, maxDelay = 10000),
        autoCreateTopics = "true",
        exclude = { IllegalArgumentException.class, NullPointerException.class }
    )
    @KafkaListener(topics = TOPIC, groupId = "${spring.kafka.consumer.group-id:banking-consumer-group}")
    public void consumeTransactionCompleted(
            String message,
            @Header(KafkaHeaders.RECEIVED_TOPIC) String topic,
            @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
            @Header(KafkaHeaders.OFFSET) long offset) {

        log.info("Received transaction event from topic: {}, partition: {}, offset: {}", topic, partition,
offset);

        try {
            TransactionCompletedEvent event = objectMapper.readValue(message,
TransactionCompletedEvent.class);

            // Day 59 Idempotency Check
            if (processedEventRepository.existsByEventId(event.getEventId())) {
                log.warn("IDEMPOTENCY TRIGGERED: Event {} already processed. Skipping duplicate.",
event.getEventId());
                return;
            }

            // Business logic (e.g. log, send notification, audit)
            log.info("Processing transaction completed for account: {}, amount: {}",
                    event.getAccountNumber(), event.getAmount());

        } catch (Exception e) {
            log.error("Failed to process message from topic: {}", topic, e);
            throw new RuntimeException(e); // Re-throw so @RetryableTopic handles retry & DLT
        }
    }

    // Day 58: Handles permanently failed messages landing in Dead Letter Topic
    @DltHandler
    public void handleDeadLetter(
```

```
            String failedMessage,
            @Header(KafkaHeaders.RECEIVED_TOPIC) String topic,
            @Header(name = "kafka_exception-message", required = false) String exceptionMsg) {

        log.error("ALERT: Message landed in DLT! Topic: {}, Reason: {}, Payload: {}",
                topic, exceptionMsg, failedMessage);
    }
}
```

2.3 Approach 2: Programmatic `DefaultErrorHandler` + `DeadLetterPublishingRecoverer`

If you prefer a single global configuration bean instead of method-level annotations, configure `DefaultErrorHandler` directly in the consumer config.

Enterprise-Digital-Banking-System / kafka / config / KafkaConsumerConfig.java — Java

```java
package org.example.enterprisedigitalbankingsystem.kafka.config;

import lombok.extern.slf4j.Slf4j;
import org.apache.kafka.common.TopicPartition;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.core.ConsumerFactory;
import org.springframework.kafka.core.KafkaOperations;
import org.springframework.kafka.config.ConcurrentKafkaListenerContainerFactory;
import org.springframework.kafka.listener.DeadLetterPublishingRecoverer;
import org.springframework.kafka.listener.DefaultErrorHandler;
import org.springframework.util.backoff.ExponentialBackOff;

@Configuration
@Slf4j
public class KafkaConsumerConfig {

    @Bean
    public DefaultErrorHandler errorHandler(KafkaOperations<Object, Object> template) {
        // 1. DeadLetterPublishingRecoverer: Appends '.DLT' to the original topic name
        DeadLetterPublishingRecoverer recoverer = new DeadLetterPublishingRecoverer(
            template,
            (record, ex) -> {
                log.error("Sending message to DLT. Topic: {}, Offset: {}, Error: {}",
                        record.topic(), record.offset(), ex.getMessage());
                return new TopicPartition(record.topic() + ".DLT", record.partition());
            }
        );

        // 2. Exponential Backoff: initial 1s, 2x multiplier, 3 retries max
        ExponentialBackOff backOff = new ExponentialBackOff(1000L, 2.0);
        backOff.setMaxElapsedTime(10000L);

        DefaultErrorHandler errorHandler = new DefaultErrorHandler(recoverer, backOff);

        // 3. Exclude non-retryable exceptions (send to DLT immediately without retrying)
        errorHandler.addNotRetryableExceptions(
            org.springframework.kafka.support.serializer.DeserializationException.class,
            IllegalArgumentException.class
        );

        return errorHandler;
    }

    @Bean
    public ConcurrentKafkaListenerContainerFactory<String, String> kafkaListenerContainerFactory(
            ConsumerFactory<String, String> consumerFactory,
            DefaultErrorHandler errorHandler) {

        ConcurrentKafkaListenerContainerFactory<String, String> factory =
                new ConcurrentKafkaListenerContainerFactory<>();
        factory.setConsumerFactory(consumerFactory);
        factory.setCommonErrorHandler(errorHandler); // Attach error handler with DLQ
        return factory;
    }
}
```

3. Day 59: Consumer Groups & Idempotency

3.1 Producer-Side Idempotency

In the banking app (`Enterprise-Digital-Banking-System`), the producer must never produce duplicate events when network retries occur.

> **How Kafka Producer Idempotence Works:** The broker assigns a unique **PID** (Producer ID) and an incremental **Sequence Number** to every message batch. If the broker receives a sequence number it already committed, it deduplicates the message and acknowledges it without creating a double entry.

Enterprise-Digital-Banking-System / kafka / config / KafkaProducerConfig.java — Java

```java
package org.example.enterprisedigitalbankingsystem.kafka.config;

import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.common.serialization.StringSerializer;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.core.DefaultKafkaProducerFactory;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.core.ProducerFactory;

import java.util.HashMap;
import java.util.Map;

@Configuration
public class KafkaProducerConfig {

    @Value("${spring.kafka.bootstrap-servers}")
    private String bootstrapServers;

    @Bean
    public ProducerFactory<String, String> producerFactory() {
        Map<String, Object> config = new HashMap<>();
        config.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
        config.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
        config.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class);

        // ═══════════════════════════════════════════════════════════════
        // Day 59: Enable Producer Idempotence & Safe Delivery Guarantee
        // ═══════════════════════════════════════════════════════════════
        config.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);      // Exactly-once per partition
        config.put(ProducerConfig.ACKS_CONFIG, "all");                    // Wait for all in-sync
replicas
        config.put(ProducerConfig.RETRIES_CONFIG, Integer.MAX_VALUE);      // Retry until broker accepts
        config.put(ProducerConfig.MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION, 5); // Maintains ordered delivery

        return new DefaultKafkaProducerFactory<>(config);
    }

    @Bean
    public KafkaTemplate<String, String> kafkaTemplate(ProducerFactory<String, String> producerFactory) {
        return new KafkaTemplate<>(producerFactory);
    }
}
```

3.2 Consumer-Side Deduplication (Idempotent Consumer)

Producer idempotence does **NOT** protect against consumer duplicates (e.g. consumer finishes sending the email, but crashes right before committing the offset back to Kafka). When a new consumer in the consumer group takes over, it re-reads that message.

The Solution: Idempotency Key Guard Table

Every event in our project already carries an `eventId` (UUID). We record each `eventId` in a `processed_events` table inside a database transaction.

Enterprise-Digital-Banking-System / kafka / consumer / entity / ProcessedEvent.java — Java

```java
package org.example.enterprisedigitalbankingsystem.kafka.consumer.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "processed_events")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ProcessedEvent {

    @Id
    private String eventId; // Unique UUID generated by Producer

    @Column(nullable = false)
    private String eventType; // "TRANSACTION_COMPLETED" or "BENEFICIARY_ACTIVATED"

    @Column(nullable = false)
    private LocalDateTime processedAt;
}
```

Enterprise-Digital-Banking-System / kafka / consumer / repository / ProcessedEventRepository.java — Java

```java
package org.example.enterprisedigitalbankingsystem.kafka.consumer.repository;

import org.example.enterprisedigitalbankingsystem.kafka.consumer.entity.ProcessedEvent;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ProcessedEventRepository extends JpaRepository<ProcessedEvent, String> {
    boolean existsByEventId(String eventId);
}
```

Consumer Implementation with Deduplication Guard

Enterprise-Digital-Banking-System / kafka / consumer / TransactionEventConsumer.java — Java

```java
@Component
@RequiredArgsConstructor
@Slf4j
public class TransactionEventConsumer {

    private static final String TOPIC = "transaction.completed";
    private final ProcessedEventRepository processedEventRepository;
    private final ObjectMapper objectMapper;

    @KafkaListener(topics = TOPIC, groupId = "${spring.kafka.consumer.group-id:banking-consumer-group}")
    @Transactional
    public void consumeTransactionCompleted(String payload) {
        try {
            TransactionCompletedEvent event = objectMapper.readValue(payload,
TransactionCompletedEvent.class);

            // Day 59 Idempotency Check:
            if (processedEventRepository.existsByEventId(event.getEventId())) {
                log.warn("IDEMPOTENCY TRIGGERED: Event {} already processed. Skipping duplicate.",
event.getEventId());
                return; // Safe exit, do not re-process!
            }

            // 1. Execute event business logic (e.g. balance check, notification)
            log.info("Processing transaction completed for account: {}, amount: {}",
                    event.getAccountNumber(), event.getAmount());

            // 2. Mark event as processed in DB
            ProcessedEvent record = ProcessedEvent.builder()
                    .eventId(event.getEventId())
                    .eventType("TRANSACTION_COMPLETED")
                    .processedAt(LocalDateTime.now())
                    .build();
            processedEventRepository.save(record);

            log.info("Successfully processed and recorded event: {}", event.getEventId());

        } catch (Exception ex) {
            log.error("Failed to process transaction event", ex);
            throw new RuntimeException(ex); // Re-throw so Kafka retry / DLQ engages
        }
    }
}
```

4. Application Properties Reference

Banking Application (Producer)

Enterprise-Digital-Banking-System / application.properties

```
# Kafka Connection
spring.kafka.bootstrap-servers=localhost:9092

# Producer Idempotence (Day 59)
spring.kafka.producer.key-
serializer=org.apache.kafka.common.serialization.
StringSerializer
spring.kafka.producer.value-
serializer=org.apache.kafka.common.serialization.
StringSerializer
spring.kafka.producer.acks=all
spring.kafka.producer.properties.enable.idempoten
ce=true
spring.kafka.producer.properties.max.in.flight.re
quests.per.connection=5
```

Notification Service (Consumer)

notification-service / application.properties

```
# Kafka Connection & Consumer Group (Day 59)
spring.kafka.bootstrap-servers=localhost:9092
spring.kafka.consumer.group-id=notification-
service-group
spring.kafka.consumer.auto-offset-reset=earliest

# Deserialization with Poison Pill Protection
spring.kafka.consumer.key-
deserializer=org.apache.kafka.common.serializatio
n.StringDeserializer
spring.kafka.consumer.value-
deserializer=org.apache.kafka.common.serializatio
n.StringDeserializer

# Manual offset commit option (optional)
spring.kafka.consumer.enable-auto-commit=false
spring.kafka.listener.ack-mode=RECORD
```

5. How to Test & Verify Days 58 & 59

Test Case	How to Perform	Expected Behavior
1. Normal Flow	Perform a deposit via `POST /api/v1/transactions/deposit`.	Message lands in `transaction.completed`. Consumer processes it and inserts into `processed_events`.
2. Duplicate Event Test (Idempotency)	In Kafka UI (`http://localhost:8085`), open `transaction.completed`, click **Produce Message**, and paste the exact same JSON payload with the same `eventId`.	Consumer logs: `IDEMPOTENCY TRIGGERED: Event ... already processed. Skipping duplicate.` No duplicate email sent.
3. Poison Pill / DLT Test (DLQ)	In Kafka UI, publish an invalid message to `transaction.completed` (e.g. `{"invalid":"corrupted"}`).	Consumer retries 3 times with exponential delays (1s, 2s, 4s). Then message automatically moves into `transaction.completed.DLT`. Main partition never stalls.

6. Common Gotchas & Best Practices

1. Avoid the Infinite Retry Loop

If you don't configure `exclude = { ... }` or add non-retryable exceptions to your error handler, a poison pill will retry forever, spamming CPU and logs. Always exclude `DeserializationException` and `IllegalArgumentException`.

2. Transaction Boundaries with Idempotency

Ensure the business operation and the `processedEventRepository.save()` happen in the same `@Transactional` method. If the email succeeds but the database fails, the transaction rolls back cleanly.

3. DLT Monitoring

A Dead Letter Queue is only useful if someone monitors it! In `docker-compose.yaml`, you already have **Kafka UI** on port `8085`. Regularly inspect topics ending in `.DLT` to catch failed events.

7. Kafka Elements, Annotations & Methods: What They Are & Why We Use Them

If you are new to Apache Kafka, this section is your comprehensive companion guide. It explains every single Kafka element, annotation, method, and configuration property used in this project so you understand exactly **what** each line of code does and **why** we use it in a digital banking system.

7.1 The End-to-End Event Flow in Our Banking App

[Customer deposits ₹5,000] → POST /api/v1/transactions/deposit │ ▼ 1. TransactionServiceImpl updates balance in MySQL & commits ledger entry │ ▼ 2. TransactionEventProducer.publish(event) - Serializes event to JSON String - Sends with Key: accountId.toString() │ ▼ 3. Apache Kafka Broker (KRaft cluster) - Topic: "transaction.completed" - Hashes Key → routes to Partition (e.g. Partition 1) - Assigns monotonic Offset (e.g. Offset 42) - Acknowledges Producer (acks=all) │ ▼ 4. TransactionEventConsumer (@KafkaListener) - Background thread polls message from Partition 1 - Checks Day 59 Idempotency: Has eventId already been processed? - If duplicate → logs warning and safely skips! - If new → executes business logic & records eventId in processed_events table - If failure occurs → Day 58 @RetryableTopic retries with backoff → moves to .DLT if exhausted

7.2 Core Architecture Elements

Element	What It Is in Kafka	Why We Use It in This Project
Topic	A named stream / channel of records. Like a mailbox or table of append-only events.	We use two dedicated topics: • `transaction.completed`: Emitted on deposit, withdraw, transfer. • `beneficiary.activated`: Emitted when a payee is approved.
Partition	A topic is split into multiple parallel ordered logs called partitions (we use 3 partitions).	**Parallelism & Throughput:** Instead of processing 1 transaction at a time, 3 consumers can process 3 partitions in parallel.
Message Key	An optional identifier sent with each record (e.g. `accountId` or `customerId`).	**Strict Ordering Guarantee:** Kafka hashes the key: `hash(key) % numPartitions`. All transactions for the same account **always go to the same partition**, guaranteeing deposits and withdrawals for Account #101 are processed in exact chronological order!
Offset	A unique sequential integer (0, 1, 2, 3...) assigned to every message in a partition.	**Bookmark / Progress Tracker:** Kafka and the consumer group remember the last committed offset, so if the app restarts, it resumes exactly where it left off without losing messages.
Consumer Group	A collection of consumer instances working together under a single group ID (`banking-consumer-group`).	**Load Balancing:** Partitions are automatically divided across instances in the group. If an instance crashes, Kafka rebalances partitions to the surviving instances.
Dead-Letter Topic (DLT / DLQ)	A companion topic (e.g. `transaction.completed.DLT`) for messages that failed all retry attempts.	**Poison Pill Isolation:** If a corrupted message fails repeatedly, sending it to DLT allows the main queue to keep flowing without halting operations for healthy customers.

7.3 Producer Classes, Methods & Properties

Producer Element	Code / Configuration	Purpose & Why We Use It
KafkaTemplate	`private final KafkaTemplate<String, String> kafkaTemplate;`	Spring's core template wrapper around the Apache Kafka `KafkaProducer`. Handles thread safety, connection pooling, and callback execution.
kafkaTemplate.send()	`kafkaTemplate.send(TOPIC, key, payload);`	Asynchronously fires the message to the Kafka broker. Does not block the HTTP request thread, returning control to the caller in < 2 milliseconds.
ProducerFactory	`public ProducerFactory<String, String> producerFactory() { ... }`	Factory bean responsible for creating raw `KafkaProducer` instances with the customized properties map.
StringSerializer	`ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class`	Kafka brokers only store raw byte arrays. This serializer turns our Java `String` key and JSON payload into network bytes.
ACKS = "all"	`config.put(ProducerConfig.ACKS_CONFIG, "all");`	**Zero Data Loss:** The broker will not acknowledge receipt until *all in-sync replica brokers* have committed the record to disk. Crucial for financial data integrity.
enable.idempotence = true	`config.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);`	**Producer-Side Deduplication:** The broker assigns a Producer ID (PID) and sequence number. If a network timeout causes the producer to resend, the broker ignores

		duplicate sequence numbers.
max.in.flight.requests = 5	`config.put(ProducerConfig.MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION, 5);`	Allows pipelining up to 5 requests at once for maximum network throughput while keeping in-order delivery guaranteed (when idempotence is enabled).

7.4 Consumer Classes, Annotations & Methods

Consumer Element	Code / Configuration	Purpose & Why We Use It
@EnableKafka	`@EnableKafka` `@Configuration` `public class KafkaConsumerConfig`	Enables Spring Kafka's background annotation-driven listener endpoint infrastructure. Without this, `@KafkaListener` methods will not be registered.
@KafkaListener	`@KafkaListener(topics = "transaction.completed", groupId = "banking-consumer-group")`	Spawns a continuous background polling loop. Whenever a new message arrives in the topic, Spring invokes this method and passes the message payload.
ConcurrentKafkaListenerContainerFactory	`public ConcurrentKafkaListenerContainerFactory<String, String> ...`	Builds the background listener containers. Allows configuring concurrency (multiple consumer threads), custom error handlers, and message converters.
StringDeserializer	`ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class`	Turns incoming raw network bytes from the broker back into readable Java `String` JSON text.
auto.offset.reset = "earliest"	`config.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");`	If a new consumer group starts with no committed offset history, `earliest` forces it to read from message 0, ensuring no historical events are lost.
@Header(...) Metadata	`@Header(KafkaHeaders.RECEIVED_PARTITION) int partition,` `@Header(KafkaHeaders.OFFSET) long offset`	Extracts transport metadata from the Kafka record headers into method variables, essential for debugging lag and tracing partition assignments.

7.5 Day 58 Error Handling & Resilience Annotations

Resilience Element	Code / Configuration	Purpose & Why We Use It
@RetryableTopic	`@RetryableTopic(attempts = "4", autoCreateTopics = "true")`	**Non-Blocking Retries:** Creates separate retry topics (e.g. `topic-retry-1000`). Failed messages move to retry topics without blocking other messages in the main topic!
@Backoff	`backoff = @Backoff(delay = 1000, multiplier = 2.0, maxDelay = 10000)`	**Exponential Backoff:** Attempt 1 waits 1s, Attempt 2 waits 2s, Attempt 3 waits 4s. Prevents overwhelming a downstream system that is already struggling.
exclude = { ... }	`exclude = { IllegalArgumentException.class }`	**Poison Pill Filter:** Exceptions that can never succeed on retry (like corrupt JSON or invalid arguments) skip retries and go directly to the Dead Letter Topic.
@DltHandler	`@DltHandler public void handleDeadLetter( ... ) { ... }`	The final destination method invoked when all 4 retry attempts have failed. Allows logging an alert, notifying devops, or writing to a failed-events audit table.
DefaultErrorHandler	`new DefaultErrorHandler(recoverer, backOff);`	The programmatic alternative to `@RetryableTopic`. Configured at the factory level to provide centralized retry and recoverer logic across all consumers.
DeadLetterPublishing Recoverer	`new DeadLetterPublishingRecoverer(template, ... )`	Helper class used by `DefaultErrorHandler` to automatically publish failed records to a destination DLT partition.

7.6 Day 59 Idempotency & Deduplication Elements

Idempotency Element	Code / Configuration	Purpose & Why We Use It
eventId (UUID)	`private String eventId; // in event DTO`	A unique identifier generated at the instant the transaction is created (e.g. `UUID.randomUUID().toString()`). Acts as the event's permanent fingerprint.
ProcessedEvent (Entity)	`@Entity @Table(name = "processed_events") public class ProcessedEvent { @Id String eventId; ... }`	A dedicated database table storing previously consumed event IDs. Serves as our persistent deduplication guard.
existsByEventId()	`if (processedEventRepository.existsByEventId(eventId)) return;`	**Idempotency Short-Circuit:** If Kafka re-delivers an event after a consumer crash or rebalance, this check detects it, logs a warning, and safely skips execution.
@Transactional Boundary	`@Transactional public void consumeTransactionCompleted( ... ) { ... }`	Ensures that executing the business action and saving the `ProcessedEvent` record succeed or fail together as an atomic database unit.